PRACTICAL 9

Aim :

Implement server-side in-memory caching with node-cache for the Task Management API. Cache repeated task reads, invalidate cached data after POST, PUT, and DELETE operations, and compare cached and uncached response behavior.

Code :

backend/cache.js :-

```
const NodeCache = require('node-cache');

const cache = new NodeCache({ stdTTL: 60, checkperiod: 120 });

const stats = { hits: 0, misses: 0 };

function get(key) {

  const value = cache.get(key);

  if (value === undefined) {

    stats.misses += 1;

    return undefined;

  }

  stats.hits += 1;

  return value;

}

function set(key, value) { cache.set(key, value); }

function invalidateTaskCaches(id) {

  cache.del('tasks:all');

  if (id) cache.del(`tasks:${id}`);

}

function getStats() {

  return { ...stats, keys: cache.keys(), ttlSeconds: 60 };
```

```
}

module.exports = { get, set, invalidateTaskCaches, getStats };
```

Code :

backend/routes/tasks.js :-

```
router.get('/', asyncHandler(async (req, res) => {
  const cachedTasks = cache.get('tasks:all');
  if (cachedTasks !== undefined) {
    return res.status(200).json(cachedTasks);
  }
  const tasks = await Task.find().sort({ createdAt: -1 }).lean();
  cache.set('tasks:all', tasks);
  res.status(200).json(tasks);
}));

router.get('/cache/stats', (req, res) => {
  res.status(200).json(cache.getStats());
});

router.get('/:id', ensureValidId, asyncHandler(async (req, res) => {
  const cacheKey = `tasks:${req.params.id}`;
  const cachedTask = cache.get(cacheKey);
  if (cachedTask !== undefined) return res.status(200).json(cachedTask);
  const task = await Task.findById(req.params.id).lean();
  if (!task) return res.status(404).json({ error: 'Task not found' });
  cache.set(cacheKey, task);
  res.status(200).json(task);
}));
```

Cache invalidation :-

```
cache.invalidateTaskCaches();
// after POST

cache.invalidateTaskCaches(req.params.id);

// after PUT and DELETE
```

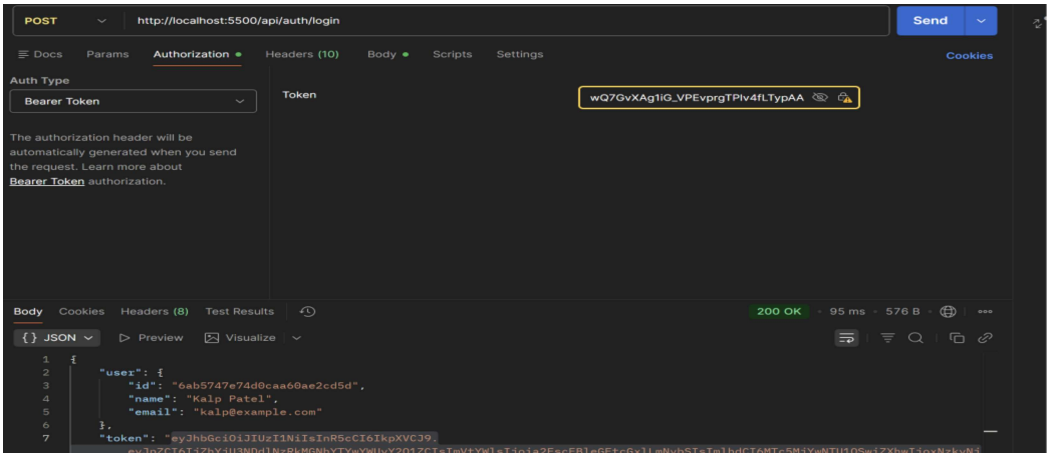
Output :

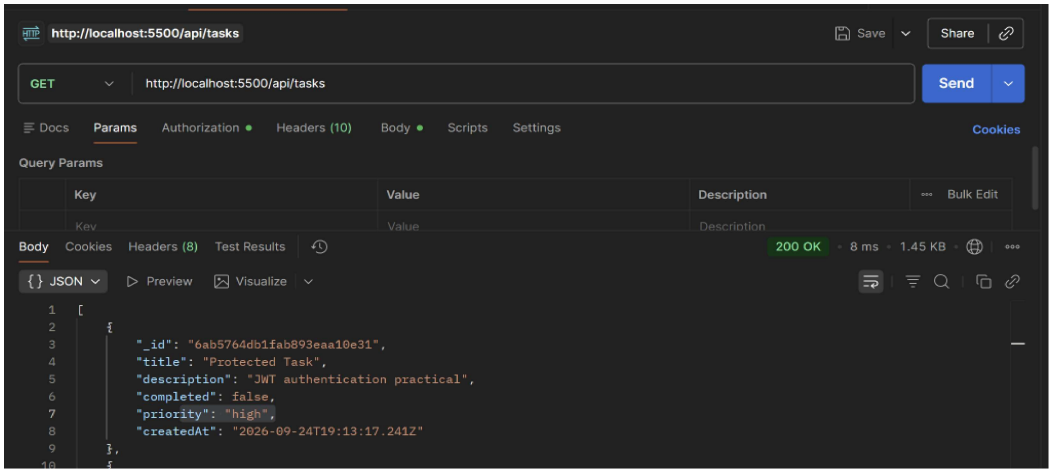
```
PS C:\Users\patel\OneDrive\Documents\SEM 5\AWD> cd "C:\Users\patel\OneDrive\Documents\SEM 5\AWD\backend"
>> npm run dev

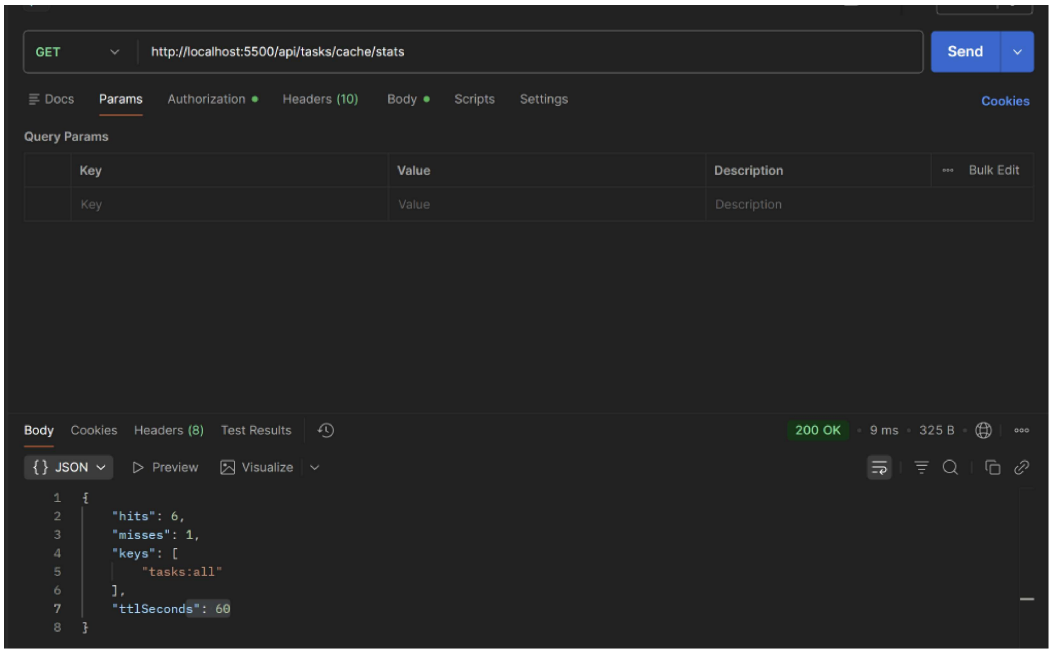
> backend@0.0.0 dev
> nodemon server.js

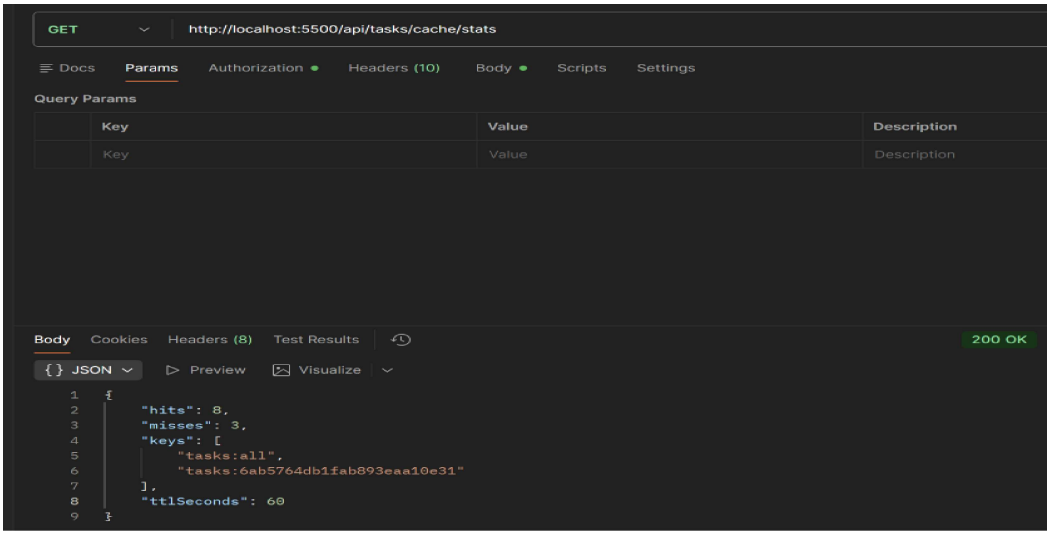
[nodemon] 2.0.22
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node server.js`
MongoDB connected
Server running on port 5500
```











Result :

In-memory caching was successfully added using node-cache with a 60-second TTL. All-task and single-task GET requests use stable cache keys, while POST, PUT, and DELETE operations invalidate stale cache entries. Cache statistics show hits and misses, and repeated reads can be compared against database-backed misses.

Response-time table :

Condition	Reading 1	Reading 2	Reading 3
Uncached	25 ms	17ms	24 ms
Cached	5 ms	2 ms	4 ms

Fill the measured values from Postman after testing.